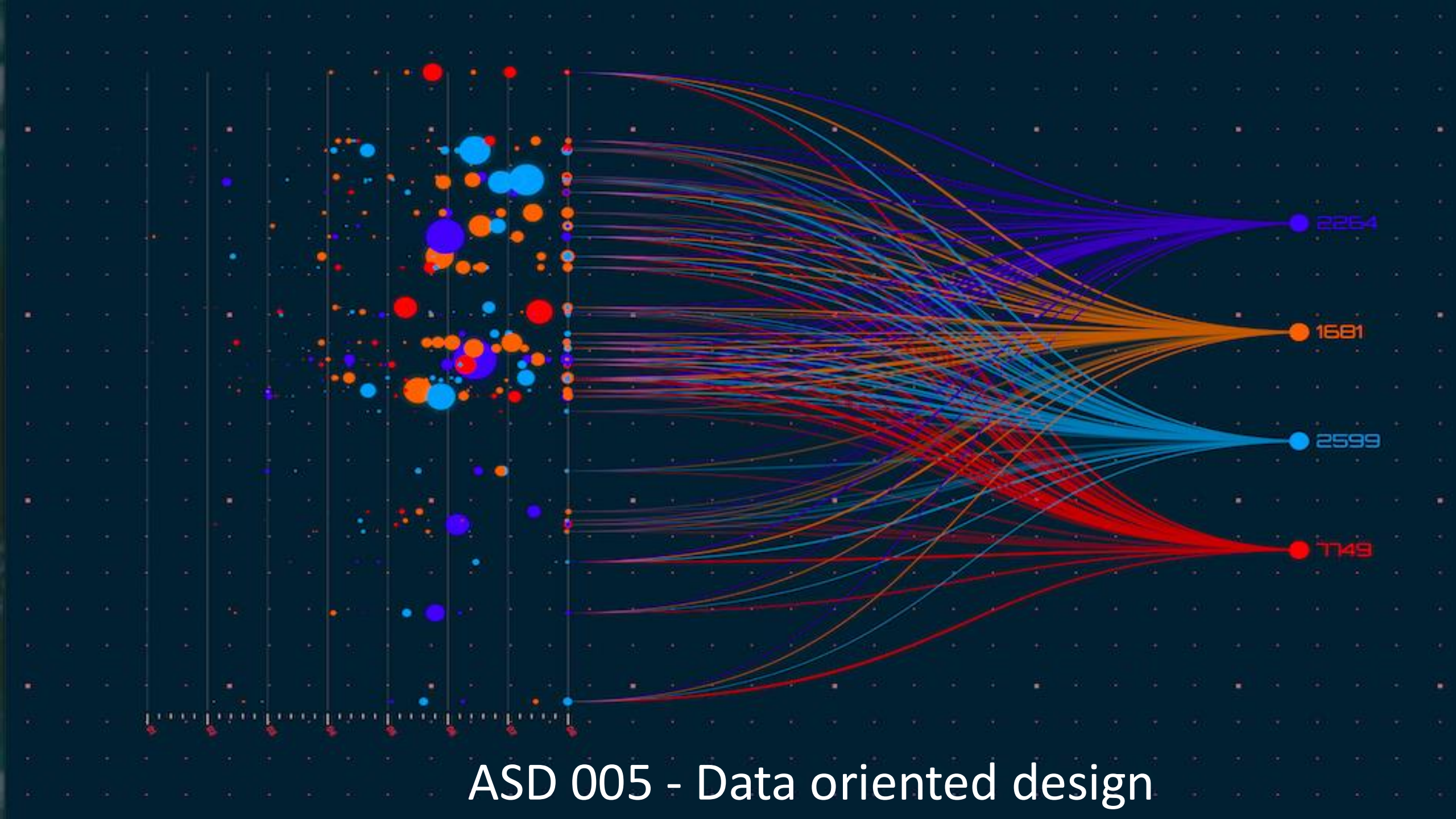


ASD 005 - Data oriented design

Advanced Software Development

Johan Bontes (johan@digitsolutions.nl)

CSC3003S



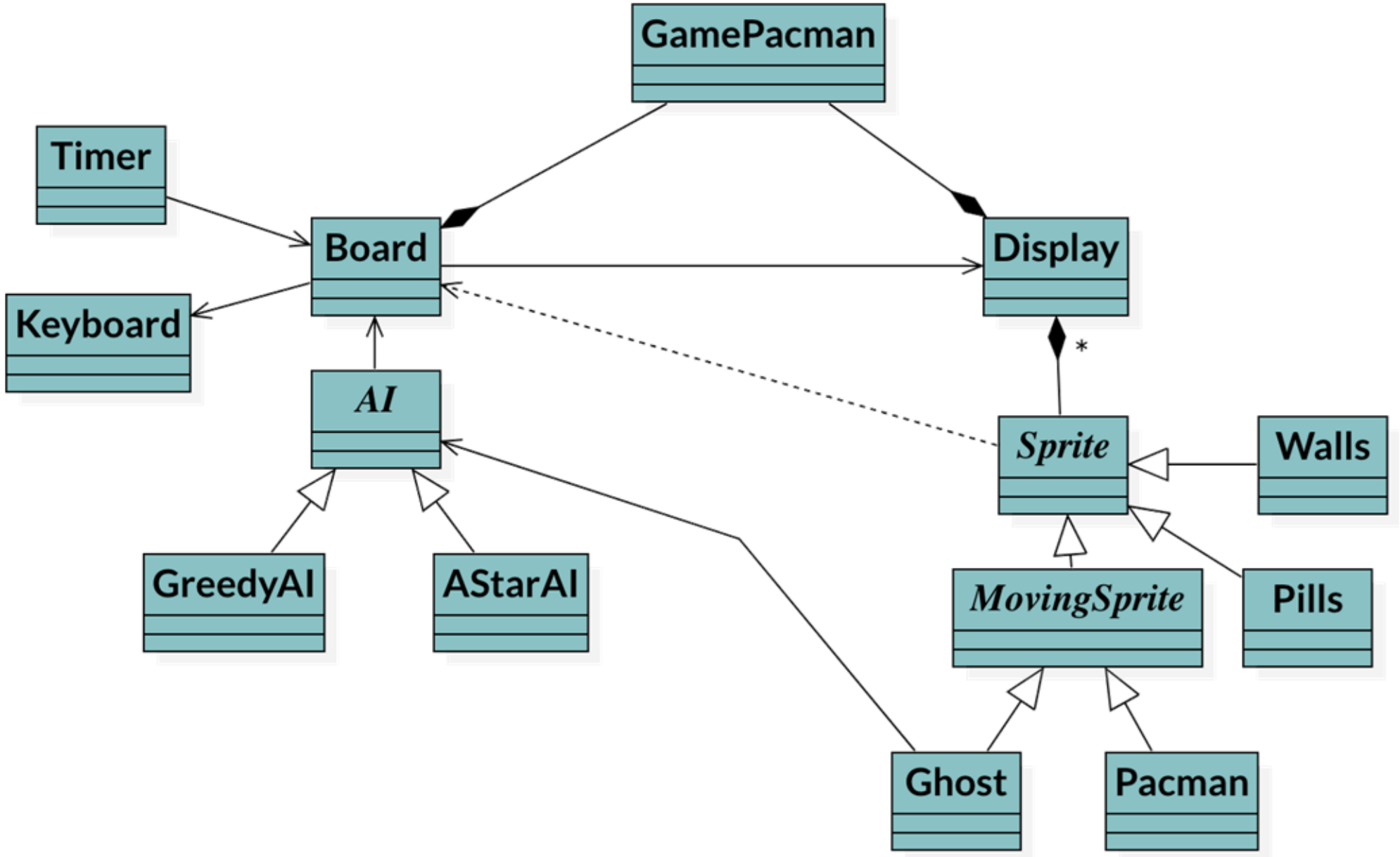
CSC3003S ASD05
Data oriented design

- Demo
- Overview
- CPU internals: data flow
- Objects data flow
- CPU internals: code flow

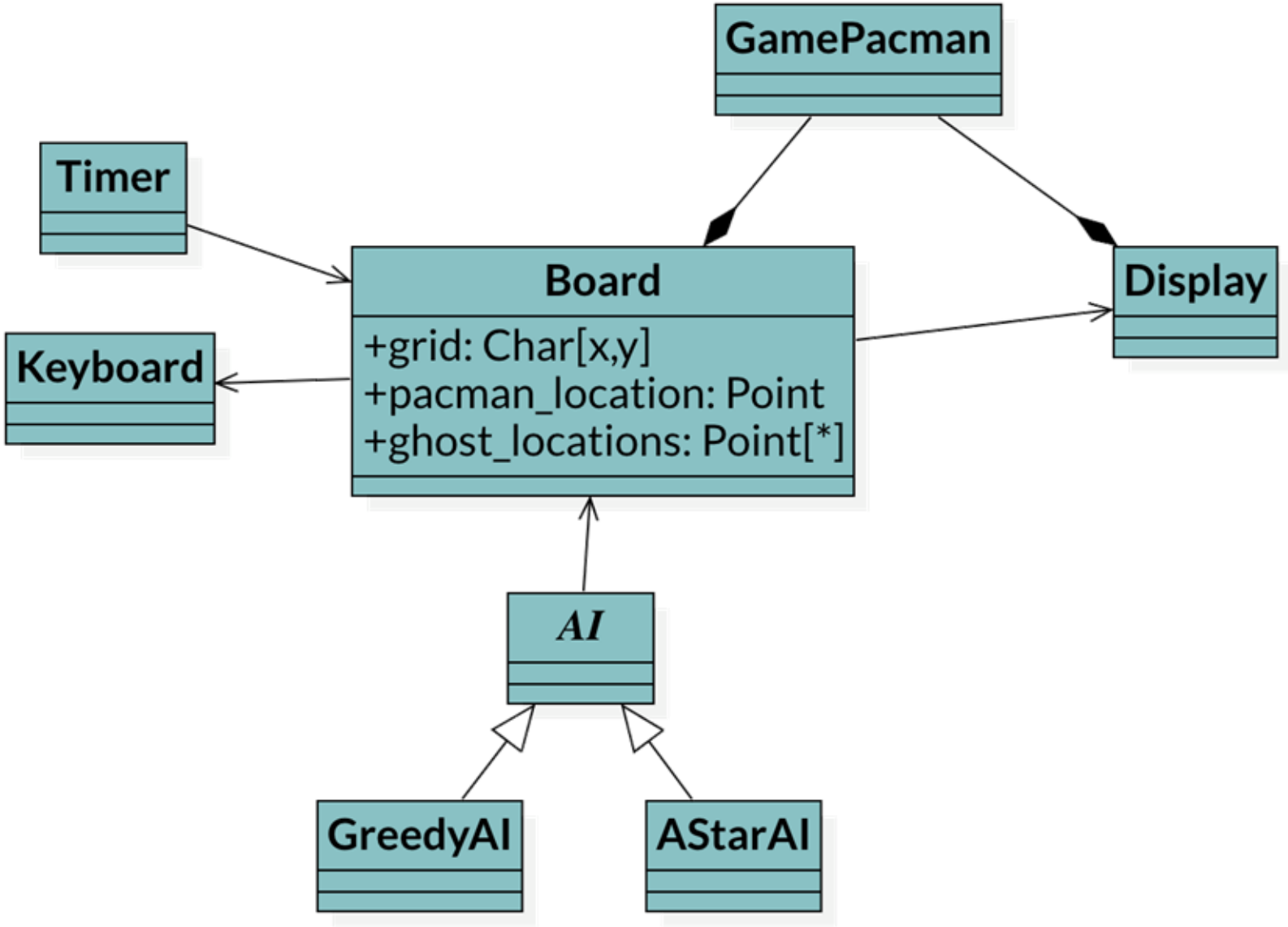
Case study 

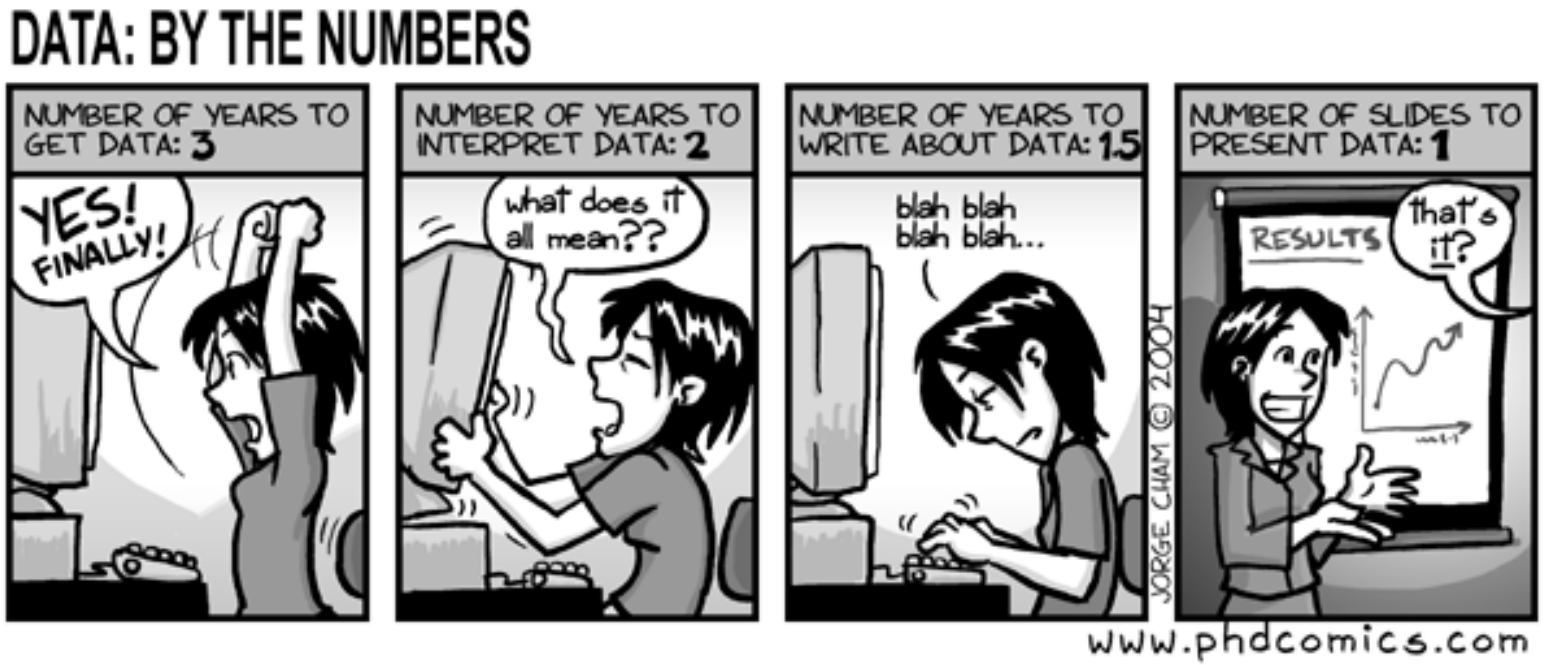
DEMO

Pacman: Traditional OOP design



Pacman: Data oriented design





CSC3003S ASD05
Data oriented design

- Demo
- Overview
- CPU internals: data flow
- Objects data flow
- CPU internals: code flow

Rule 1: If it ain’t broke…

Data oriented design

- Technique to speed up data processing
  - It’s all about data
  - Lots of data
- You no lots data -> you no DOD
- Consider using a database
- Requires you to think like a CPU
  - What slows it down?
  - What speeds it up?

Where is DOD used?

- Inside database engines
- Games
- Numerical simulation (number crunching)
- Solvers

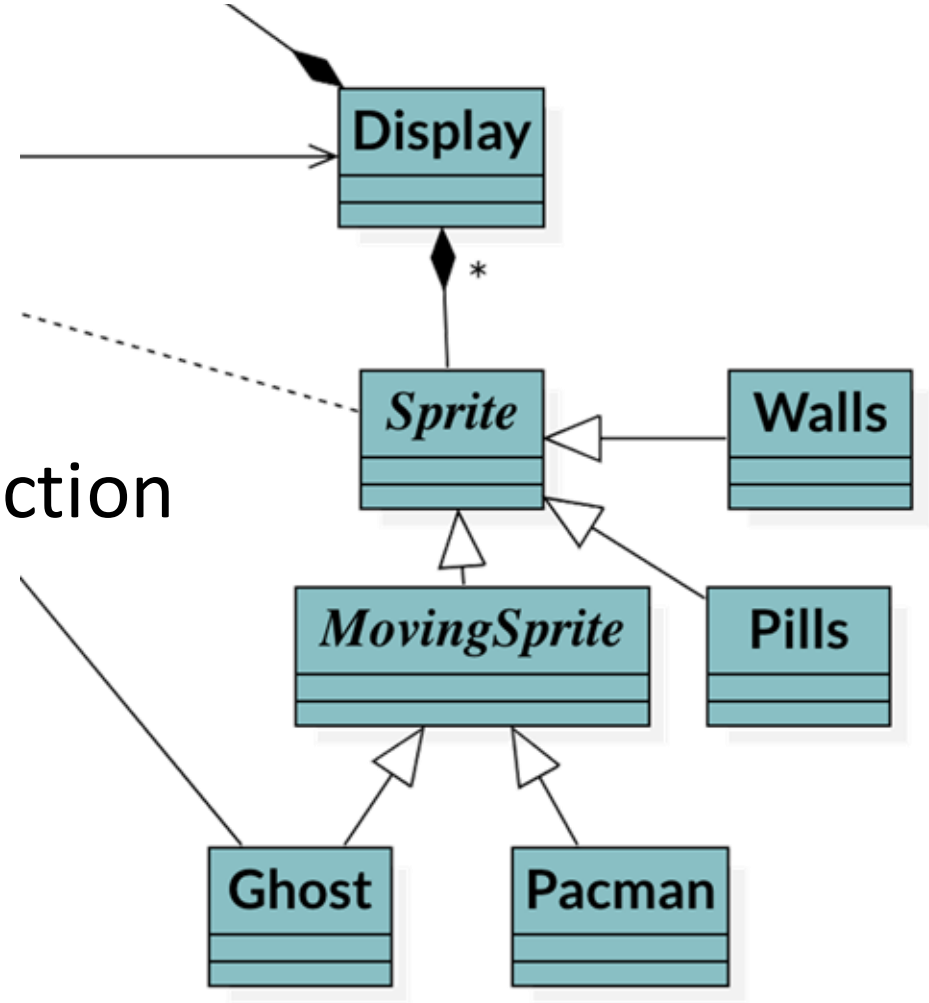
Lots of uniform data that needs to be processed

- For some reason you cannot use a database
- Database techniques are moving into game engines
  - e.g.: SpacetimeDB: https://spacetimedb.com/

It’s all about the data

Traditional OOP courses say:

- map real world entities into objects
  - solve issues using more objects
- real world = complex
  - 1-1 mapping creates blob objects
  - correct OOP mapping: lots of indirection
    - extra work for CPU: slow
    - objects live on the heap
    - mixes hot and cold data
    - mixes RO and RW data



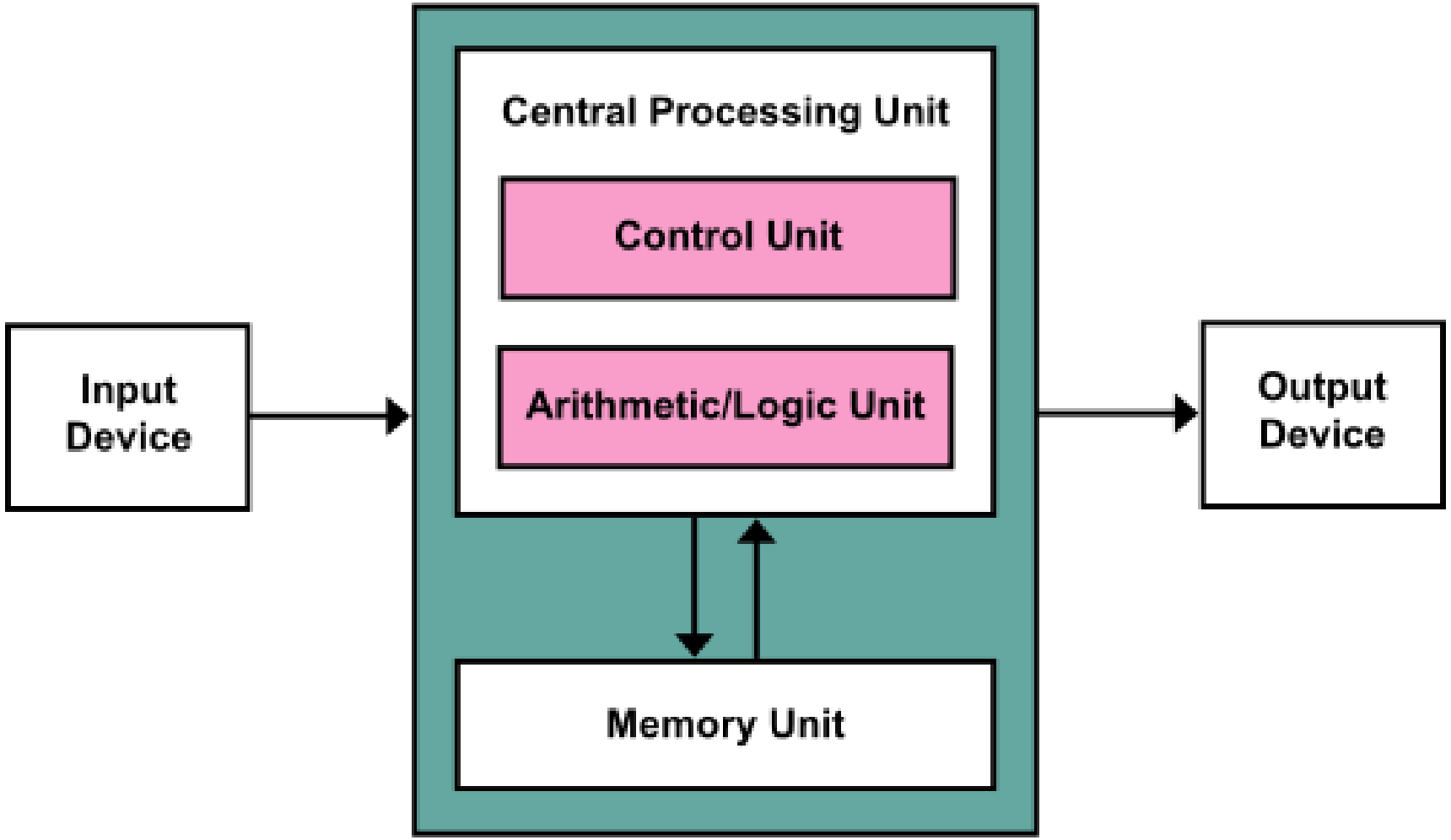


CSC3003S ASD05
Data oriented design

- Demo
- Overview
- CPU internals: data flow
- Objects data flow
- CPU internals: code flow

How does a CPU work?

CPU is a Von-Neumann machine

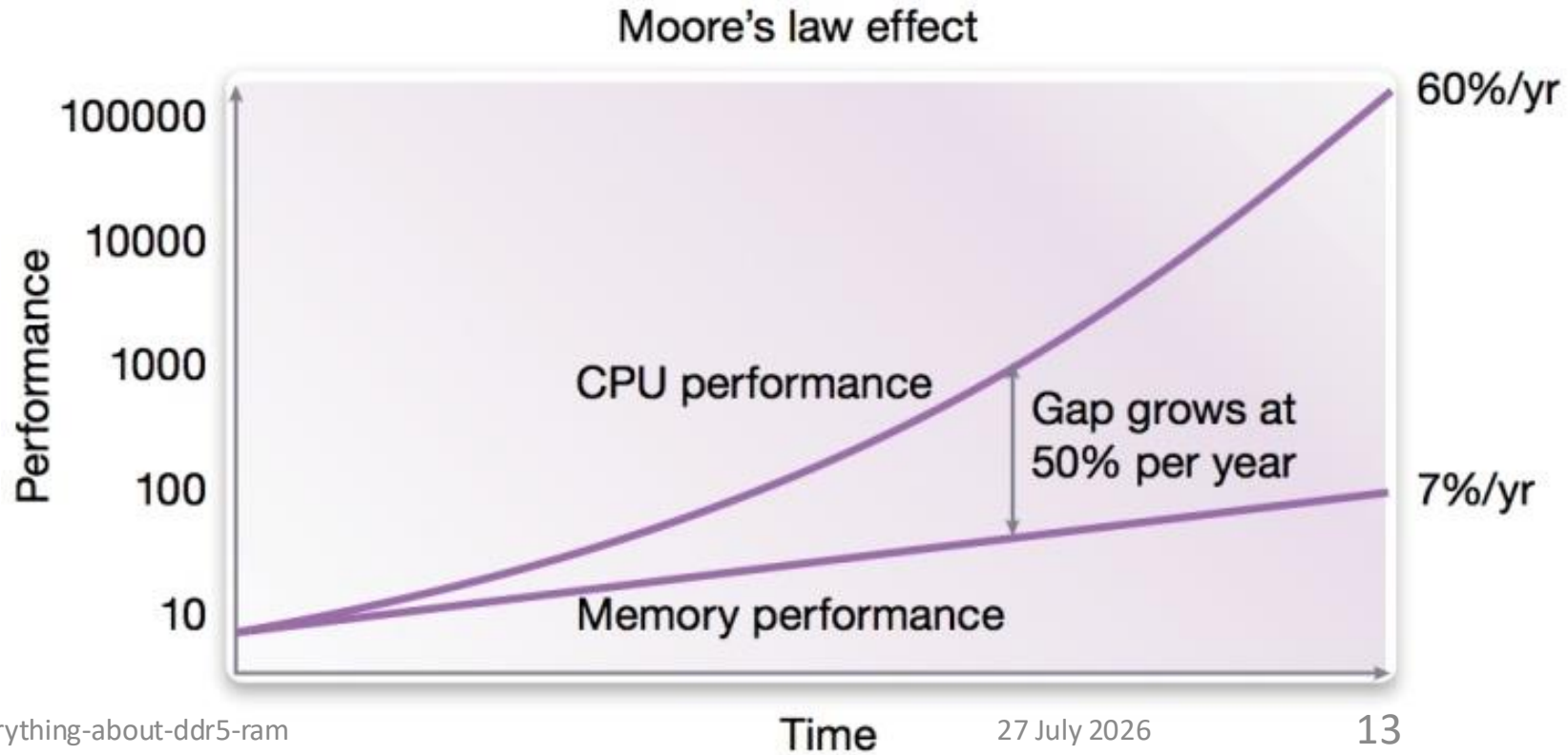


CPU: feeding the beast

5Ghz CPU can process 5G * 32 * 4 ≈ 600 GiB/s per core!

DDR5-5600: Sequential: 70 GiB/s ~10× slower
DDR5-5600: Random access: ~400 MiB/s ~1500× slower

*More cores: 8 core CPU: 600*8 = 4.8 TiB/s*



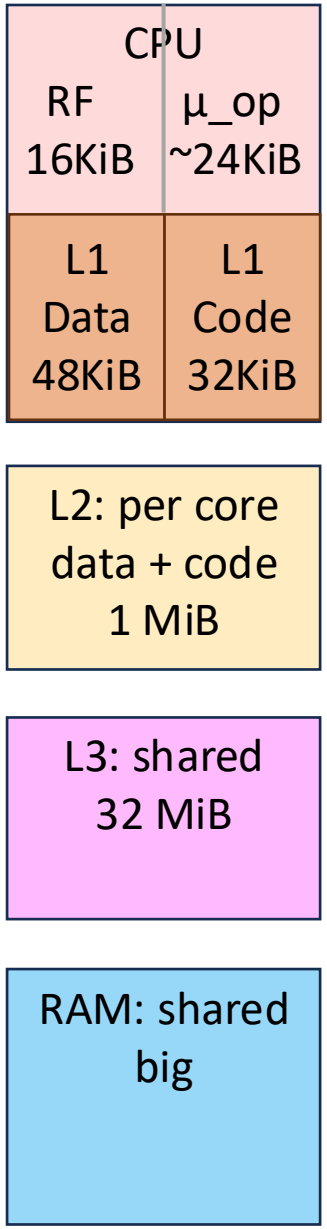
https://www.crucial.com/articles/about-memory/everything-about-ddr5-ram

The cache hierarchy

Most time is spent on hot 🔥 data

- small amount of data used over and over
  - *lookup tables*
  - *index in a database*
  - *sprite coordinates in games*
- local variables on the stack
  - scratchpad 📝
- data inside loop

Hottest data is kept in cache close to CPU



CPU 'L0:' register file + microOp cache: 600 GiB/s

200 GiB/s

20 GiB/s

2 GiB/s

0.4 GiB/s (best case)

https://www.numberworld.org/blogs/2024_8_7_zen5_avx512_teardown/
https://hardwaretimes.com/amd-zen-vs-zen-2-vs-zen-3-vs-zen-4-vs-zen-5-core-architecture-road-to-ryzen-9000-cpus/
https://en.wikipedia.org/wiki/Zen_5

Hot data?

The CPU itself has a small notepad

- For data: register file: 500 variables



And a small ‘screen’ to view code

- for instructions: 6000 micro-ops

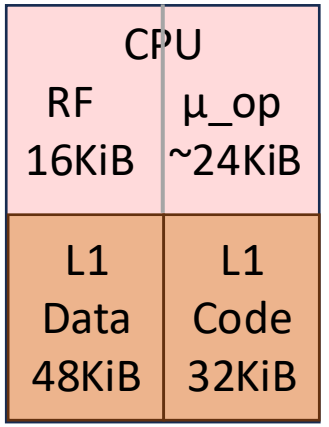
```
9
10  exposed Function get todayBookings() : cs.ReservationSelection
11      var $result : cs.ReservationSelection
12
13      If
14      $result:=This.reservationsDeparted.query("departure_date = :1"; Current date)
15      $result:=$result.or(This.reservationsArrived.query("arrival_date = :1"; Current date))
16      return $result.orderBy("todayTaskTime asc")
17
18  exposed Function getTodayBookings()->$result : cs.ReservationSelection
19      $result:=This.reservationsDeparted.query("departure_date = :1"; Current date)
20      $result:=$result.or(This.reservationsArrived.query("arrival_date = :1"; Current date))
21      $result:=$result.orderBy("todayTaskTime asc")
22
23  exposed Function get todayBookingsQty()->$result : Integer
24      $todaybookings:=This.getTodayBookings()
25      $result:=$todaybookings.length
26
27      //MARK: should be done directly in the studio
28  exposed Function getAllCars() : cs.CarSelection
29      return This.cars
```

The problem with cache

CPU cannot use RAM directly!

- Data: only if in register file
- Code: only if in µ_op file


New data in -> old data out

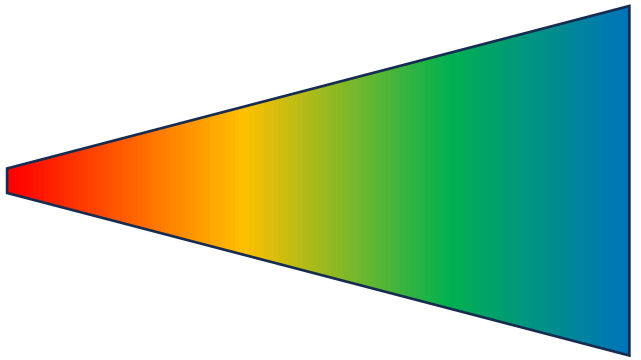
Data in/out CPU goes via L1

Data in L1 goes via L2

Data in L2 goes via L3

The closer to the CPU, the hotter the data

Also: more re-used/recent = hotter



Eviction schemes

Older CPUs:

- 2000's LRU least recently used (delete oldest)
- 2010's LFU least frequently used (delete coldest data)

Modern CPUs use a (tiny) neural network
mix between LRU/LFU

Modern CPU predicts which data/code is needed next

- Speculative prefetching (security)
- Tip 💡: make your data use predictable

Access patterns

DDR5-5600: Sequential: 70 GiB/s ~10× slower
DDR5-5600: Random access: ~400 MiB/s ~1500× slower

```
final int GB = 1024 * 1024 * 1024;
int[] array[1*GB];
```

A:
```
for (var a: array) { //sequential
  doStuff(a);
}
```

B:
```
r = new Random();
for (var i = 0; i < GB; i++) {
  var i2 = r.nextInt(GB);
  a = array[i2]; //random
  doStuff(a);
}
```

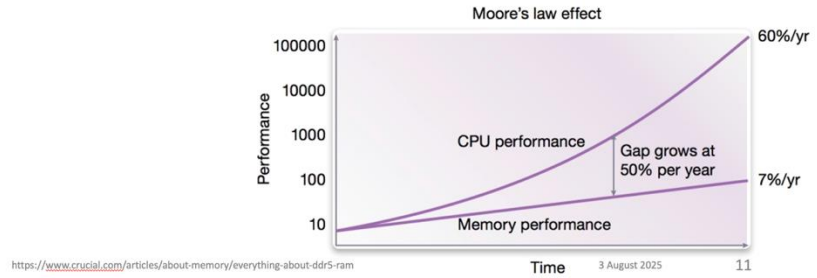
Demo

CPU: feeding the beast

5Ghz CPU can process 5G * 32 * 4 ≈ 600 GiB/s per core!

DDR5-5600: Sequential: 70 GiB/s ~10× slower
DDR5-5600: Random access: ~400 MiB/s ~1500× slower

*More cores: 8 core CPU: 600*8 = 4.8 TiB/s*



Java OOP access patterns

```
final int GB = 1024 * 1024 * 1024;
int[] array[1*GB]; //4GiB;
```

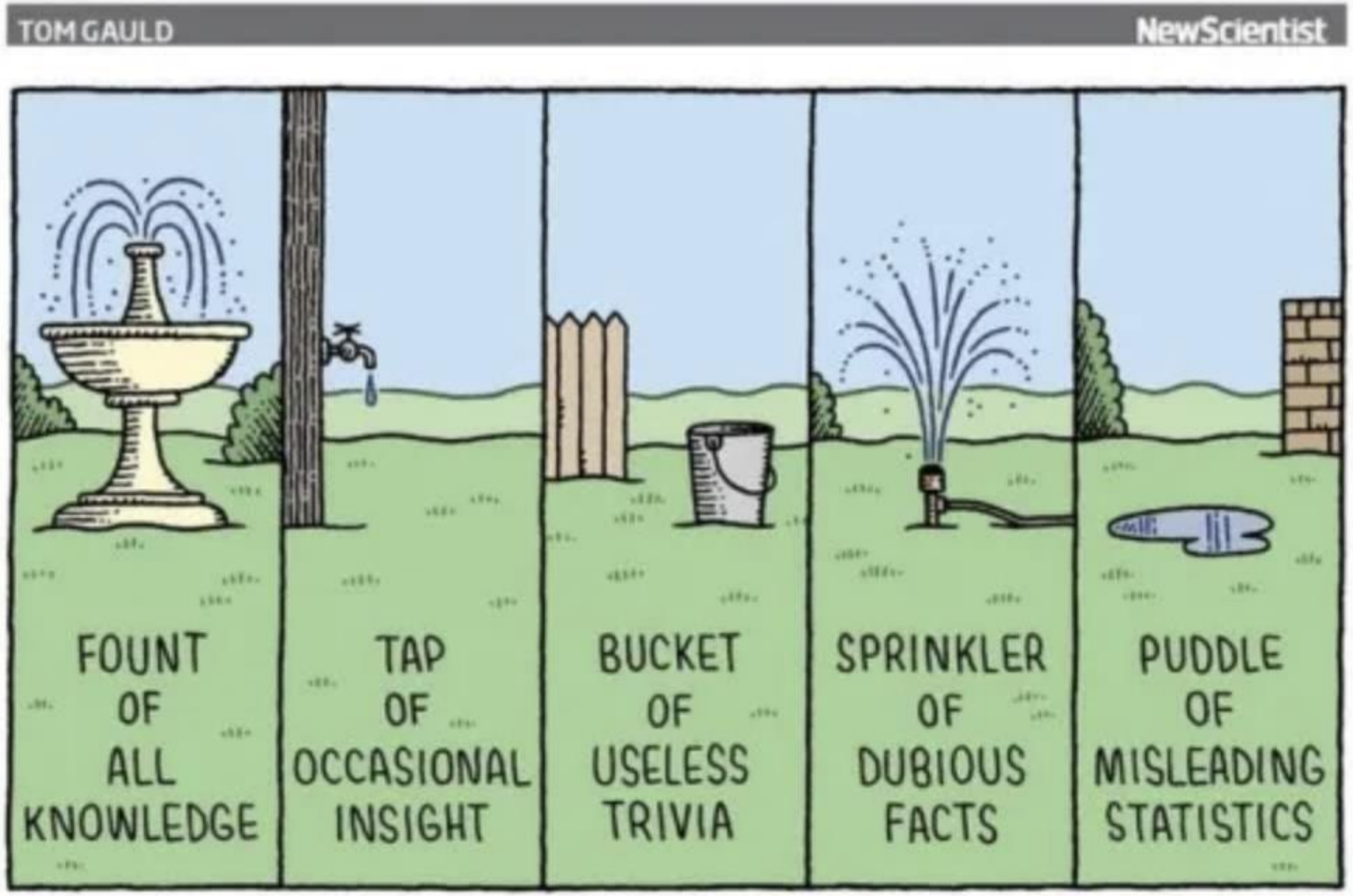
A:

```
for (var a: array) { //sequential
  doStuff(a);
}
```

B:

```
Integer[] array2[1*GB]; //48 GiB (after init) -> why?
...init all of array2
for (var a: array2) { //random access! -> why?
  doStuff(a);
}
```

Demo



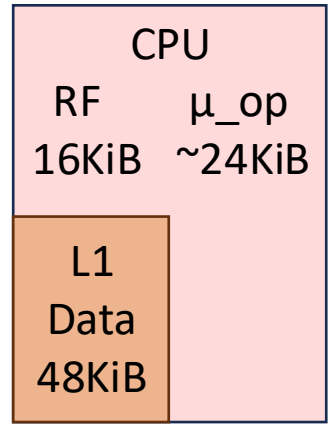
ASD 05
Data oriented design

- Demo
- Overview
- CPU internals: data flow
- Objects data flow
- CPU internals: code flow

Anatomy of an object

```
ObjectX a; //local variable L1/CPU

a = new ObjectX(); //ObjectX in Heap
```



Local variable: stack (always in L1)

Heap: object store in main memory

Object = data + code

a = link (pointer) to the heap object (8 bytes)

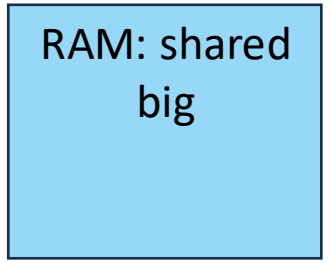
```
ObjectX = struct {
```

VMT: *'this' pointer to virtual method table (list of code)*

Data: *data members*

padding: *empty space to make size multiple of 8 bytes*

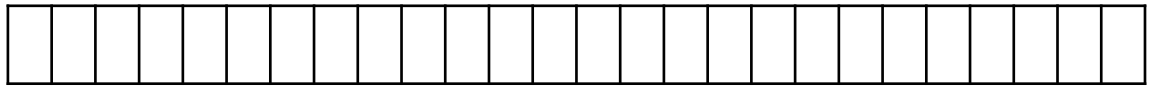
```
}
```



The heap

The heap consists of 4 lists

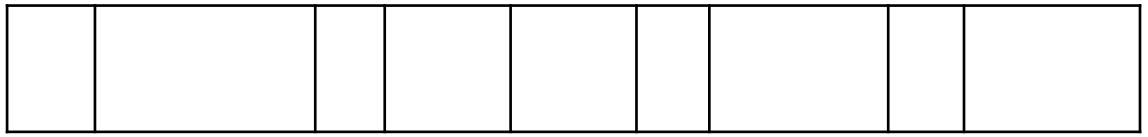
Tiny objects



Small objects

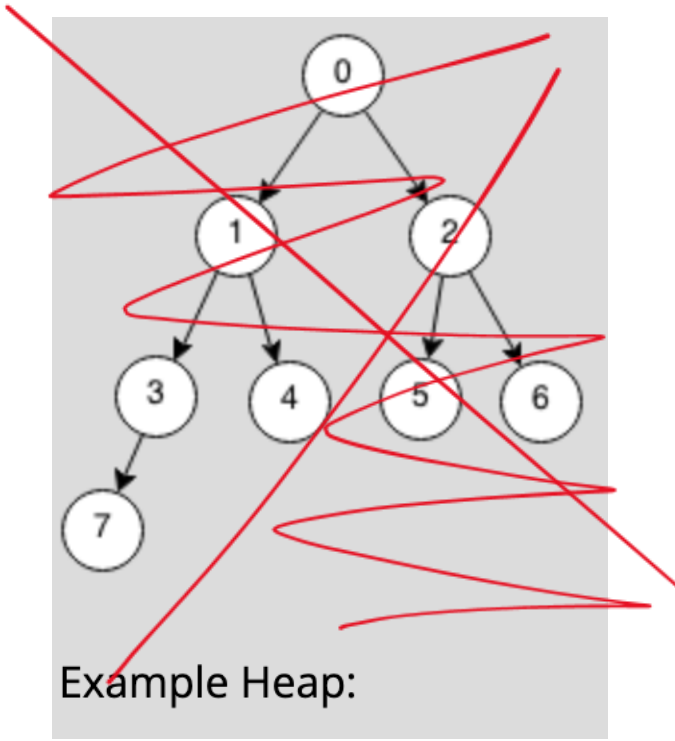


Medium objects



Large objects





Allocation: first come-first served

Destruction: random times (garbage collection)

Causes objects to be randomly distributed throughout memory

Heap bookkeeping also takes up space

Objects problem 1: they live on the heap

Object data + code is spread randomly over memory

Related objects are not stored sequentially

The longer the program runs, the more randomly spaced

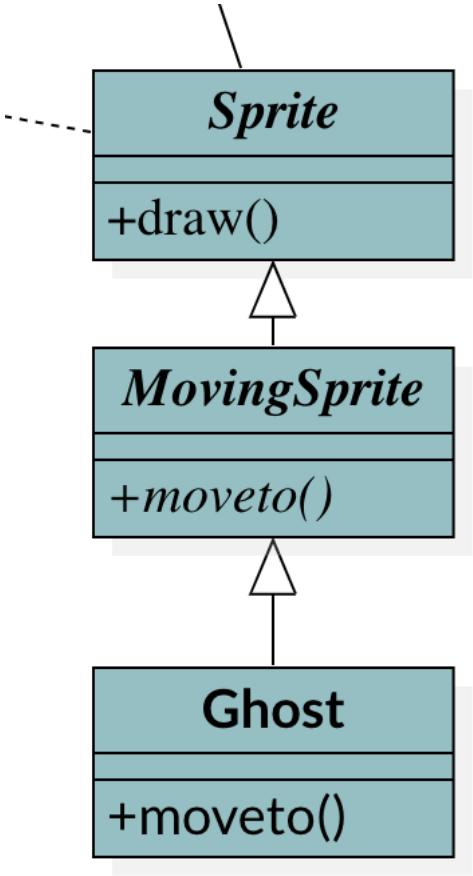
Adds 1500 cycle startup penalty to each object access

- CPU core could have performed 6000 calculations!
- Much worse delays in multi-threaded code

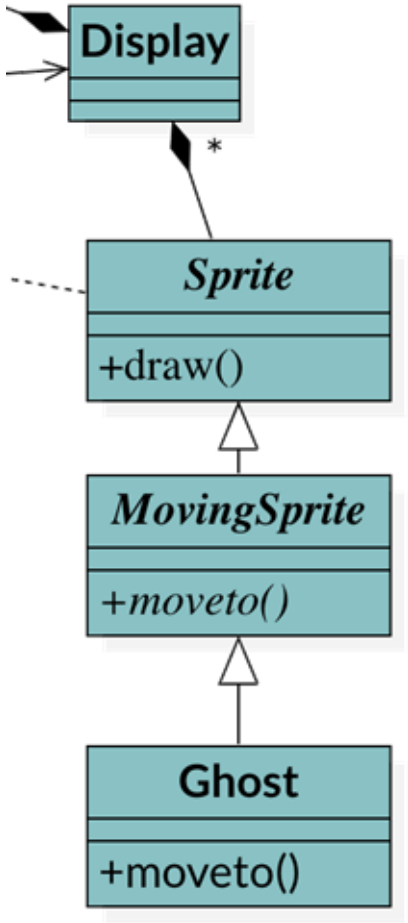
Objects problem 2: virtual methods

```
Sprites[5] dolls;
for (var i:1..4) { dolls[i] = new Ghost(); }
dolls[0] = new Pacman();
//... game loop
  dolls[i].moveto(x,y);
```

1. Lookup correct doll in the heap
2. `moveto = doll.VMT[42]` -> pointer to code
3. `call moveto` -> call cannot be predicted!
   it might be a ghost, or pacman, or zombie3, or ...
   all move in different ways



Objects problem 2: virtual methods




```
public void Ghost.draw() {
  bitmap = getScaryBitmap();
  super.draw();
}

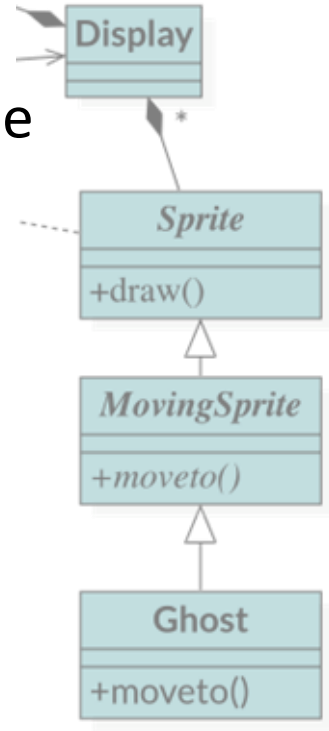
public void MovingSprite.draw() {
  location = point(x,y);
  super.draw();
}

Sprite.draw() {
  display.draw(bitmap, location);
}
```

Objects problem 2: virtual methods

```
Loop: dolls[i].draw();
```

1. Lookup correct doll in the heap
2. draw = doll.VMT[41] -> pointer to code
3. call Ghost.draw()
4. parent = doll.VMT[0]
5. draw = parent.VMT[41]
6. call MovingSprite.draw()
7. grandparent = parent.VMT[0]
8. draw = grandparent.VMT[41]
9. call Sprite.draw()
10. display = lookup display object in heap
11. draw = display.VMT[125]
12. call Display.draw(bitmap, location)
13. bitmap = lookup bitmap object in heap

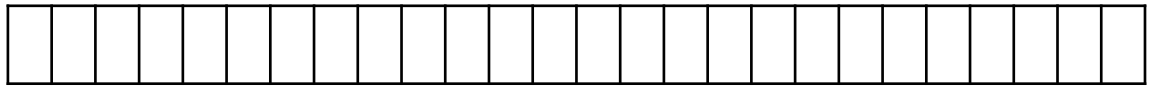



```
public void Ghost.draw() {
  bitmap = getScaryBitmap();
  super.draw();
}
public void MovingSprite.draw() {
  location = point(x,y);
  super.draw();
}
Sprite.draw() {
  display.draw(bitmap, location);
}
```

The heap

The heap consists of 4 lists

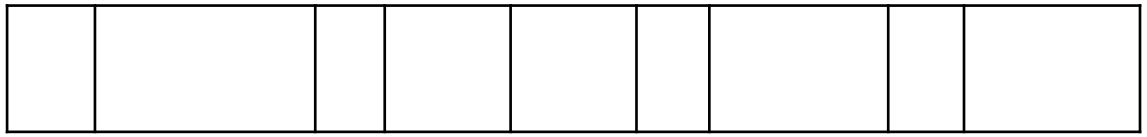
Tiny objects



Small objects



Medium objects



Large objects



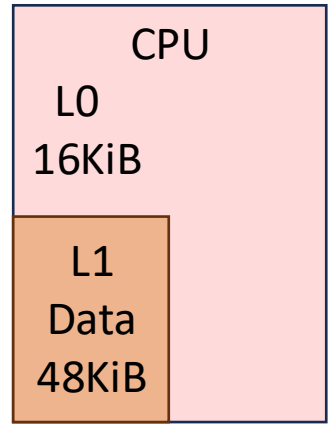
Allocation: first come-first served

Destruction: random times (garbage collection)

Causes objects to be randomly distributed throughout memory

Heap bookkeeping also takes up space

Problem 2a: data cache eviction

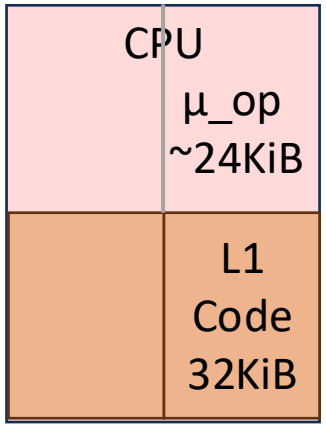



```
Loop: dolls[i].draw();
```

1. Lookup correct doll in the heap
2. `draw = doll.VMT[41]` -> pointer to code
   `call Ghost.draw()`
3. `parent = doll.VMT[0]`
4. `draw = parent.VMT[41]`
   `call MovingSprite.draw()`
5. `grandparent = parent.VMT[0]`
6. `draw = grandparent.VMT[41]`
   `call Sprite.draw()`
7. `display = lookup display object in heap`
8. `draw = display.VMT[125]`
   `call Display.draw(bitmap, location)`
9. `bitmap = lookup bitmap object in heap`

problem 2b: code cache eviction

```
Loop: dolls[i].draw();
    Lookup correct doll in the heap
    draw = doll.VMT[41] -> pointer to code
a. call Ghost.draw() //cannot be predicted
    parent = doll.VMT[0]
    draw = parent.VMT[41]
b. call MovingSprite.draw() //maybe predicted
    grandparent = parent.VMT[0]
    draw = grandparent.VMT[41]
c. call Sprite.draw() //maybe predicted
    display = lookup display object in heap
    draw = display.VMT[125]
d. call Display.draw(bitmap, location) //maybe predicted
    bitmap = lookup bitmap object in heap
```



Why cache eviction?

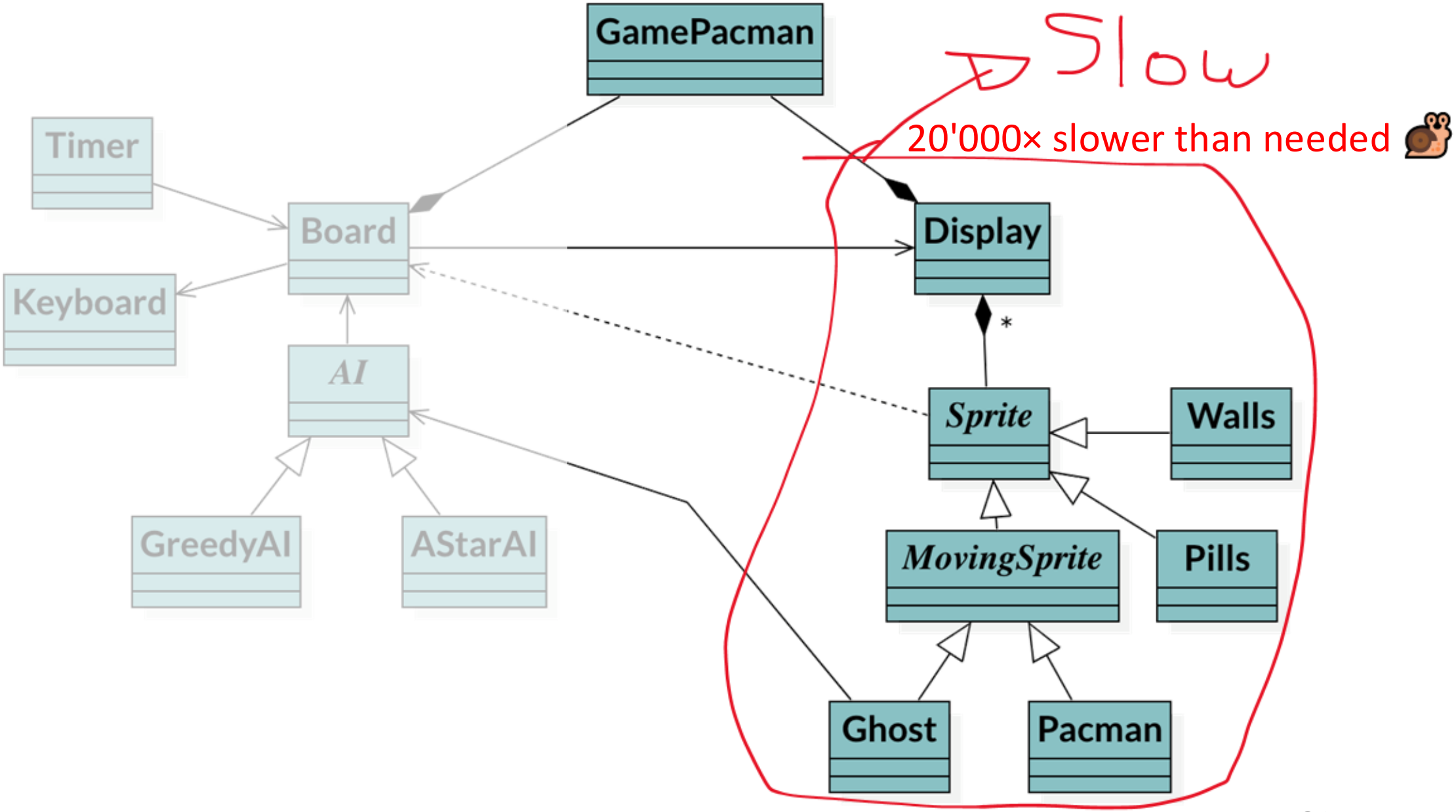
Minimum load size for data

- 32 bytes on AMD/Intel
- Even if you ask for 1 byte, 32 bytes will be fetched
  - Thus 32 other bytes will be evicted.
- Why?
  - Allows hardware to fetch 256 bits in parallel

    32× faster than fetching 1 byte at-a-time

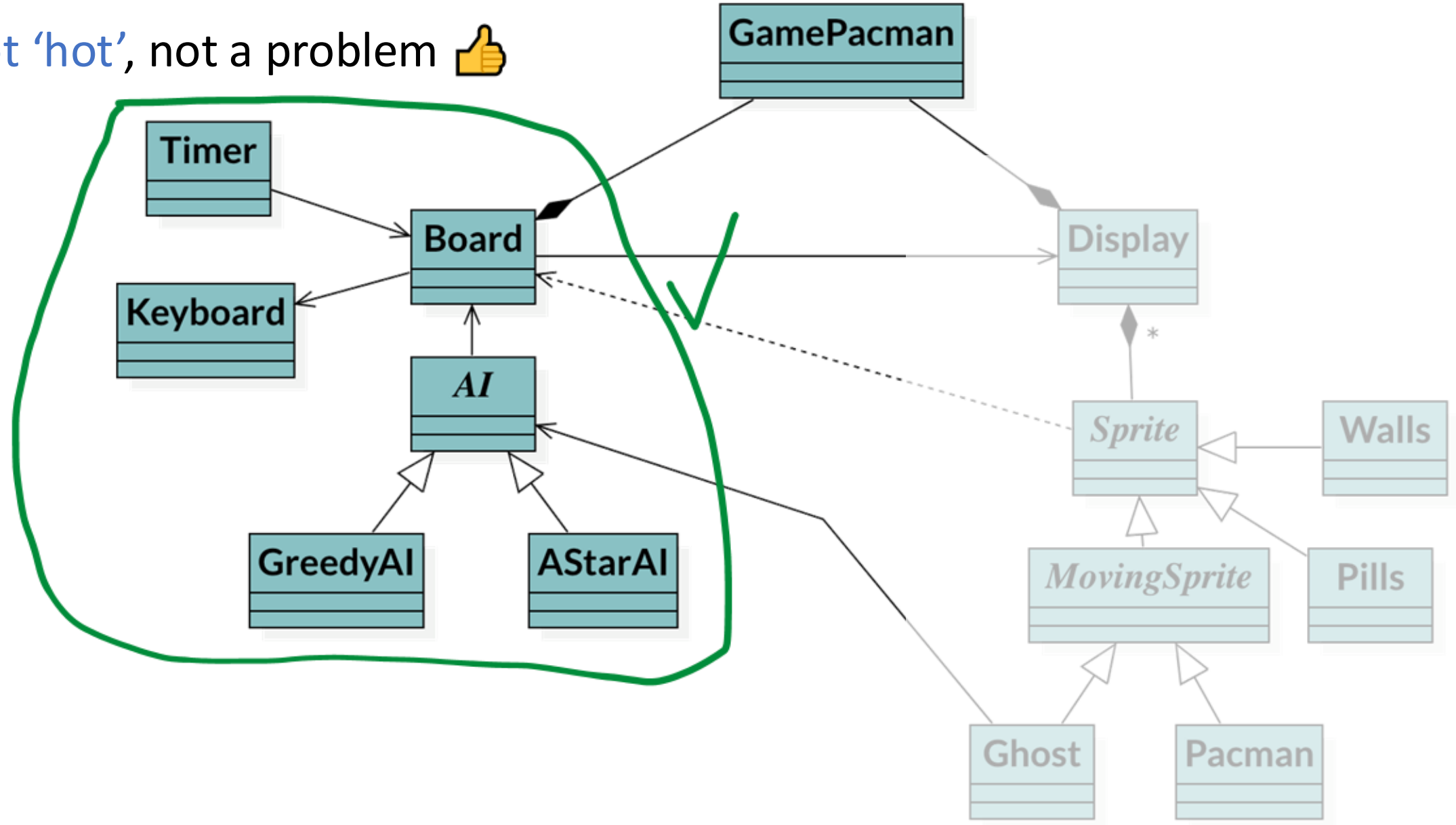
Every speed hack on the CPU has a drawback / causes problems elsewhere

Pacman: Traditional OOP design



Pacman: Traditional OOP design

Not ‘hot’, not a problem 👍



DOD step 1

- Split program into hot and cold code
- hot 🔥
  - always in a loop
  - gets executed many times per second
  - more code/objects = slower program

- cold 🥶
  - gets executed rarely
  - more code/objects = no measurable effect

Recap

Java: all Objects:

```
ClassX myobject = new ClassX();
```

anything new lives on the heap

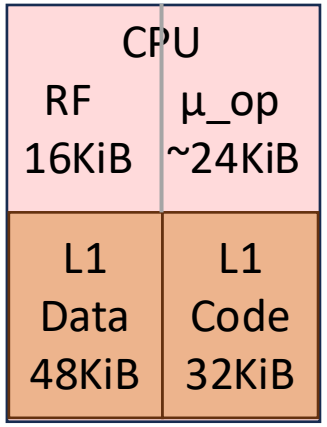
myobject is a link (*pointer*) to the data on the heap — indirect 1

All (non-static) methods are accessed indirectly. — indirect 2

```
public class ClassX {
  Bear bear;   //bear is a link to a Bear object on the heap
  //more data
}
```

indirect 3

Stack



Direct access

Local variables + links to Obj.

Fast (literally 0 time)

Tiny

Literal

Heap

RAM: shared
big
slow!
1500 cycles

Indirect access

Object data

slow: up to 1500 cycles

Huge

Polymorphic

Polymorphism

In Java there is only run-time polymorphism

- aka Late binding
- Very flexible
    - But comes at a cost of up to 1500 cycles per access
    - CPU cannot predict

C++ also has compile time polymorphism

- aka early binding
- Almost as flexible as Java
    - but up to 1000x faster
- Not easy (better in C++23).

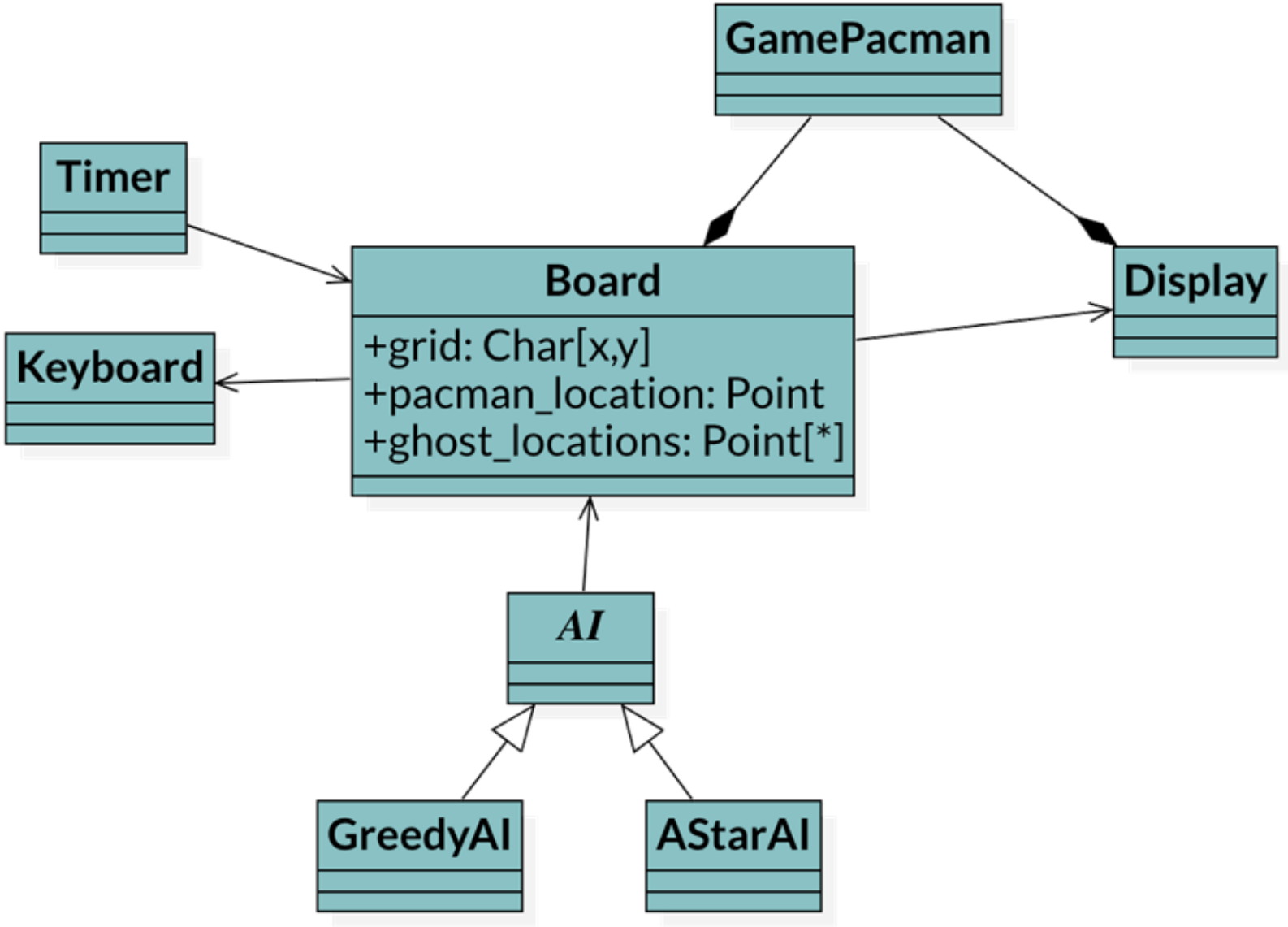
DOD step 1:

Unless 100% sure:

- Measure where the bottlenecks are
  - Instrumenting profiler
    - Java: - Java Flight Recorder
      - Visual VM
      - Async Profiler
    - C++: https://hackingcpp.com/cpp/tools/profilers.html
    - Python: https://pypi.org/project/pyinstrument/
  - Timer

```
start = hiresTimer.microsecs();
//dostuff()
end = hiresTimer.microsecs();
duration: uint64 += end – start;
```

Pacman: Data oriented design



DOD step 2

Analyse the data flow

Pacman:

Timer: 60x per sec

- where are dolls: pacman + ghosts
- check for collisions pacman vs ghosts
- check for collisions pacman vs pills/walls
- try move dolls 1 space
- animate dolls + draw grid

Data: position, bitmaps, grid

If 1000’s of ghosts, maybe pull AI in as well.

DOD step 3

Data that flows together stay together

Creative process

Arrays are your friend,

Java: arrays of simple types 👍 `char, int, float`

fewer objects 🐌 `Integer, String, ObjectX`

Abusing a `String` as an array (see Pacman demo) is OK

Only use `static` methods - pass data as first parameter

- or make data static (arrays of stuff)

C++: use `class`, but avoid `virtual`.

💡 `template` allows compile time polymorphism

Python: use `lists` of simple types, avoid `sets` and `dictionaries`

Pacman Java example

```
public class GameBoard {
  //everything static, few Objects
  public static final int PM_id = 0;
  public static final int GCOUNT = 4;
  private static int position[GCOUNT+1][2]; //0 = PM 1..4 = ghosts, xy
  private static char grid[GRID_SX][GRID_SY];
  private static char bitmaps[GCOUNT+1][BITMAP_SIZE];
  private static char doll_state[GCOUNT+1];
  private static int pillsLeft;
  private static int PM_PowerUpTimeLeft;
  private static AI GameAI; //access to normal objects
  public static void moveto(int doll_id, int x, int y);
  public static void draw(int doll_id);
}
```

Put dynamic data in another Object, not inside the DOD object.

Java: what's in an object

Mostly links, not much data

Directly included:

- simple types (`int`, `double`, `...`)
- arrays of simple types
- `enum`
- `static` code

Indirectly linked

- any `Object`
- non-static code (virtual)

Java: what's in an object

Mostly links, not much data

Directly included:

- simple types (`int, double, ...`)
- arrays of simple types
- `enum`
- `static` code

Indirectly linked

- any `Object`
- non-static code (virtual)

Fully polymorphic

Not very flexible

Solution: C++: variant objects

```
class Doll {
  boolean isGhost;
  //data..
```

Java (B) tag bits

```
isGhost = dollIdx & 1;
dollId = dollIdx >> 1;
```

`C++: std::variant`

DOD speed up

Static methods avoid indirect code lookup

Time saved:

20 x 1500 = 30K x 5 dolls = 150K cycles / 5Ghz

~ 30 µs per frame
More like ~300 µs per frame (due to other slowdowns)

Not worth it for Pacman per sé.

But if many ghosts, frame rate may drop below playable.

Searching for patterns in Conway's Game of Life



DOD speedup: 1 000 000x

Data parallelism: 1 000x

Total speedup: 1 000 000 000x

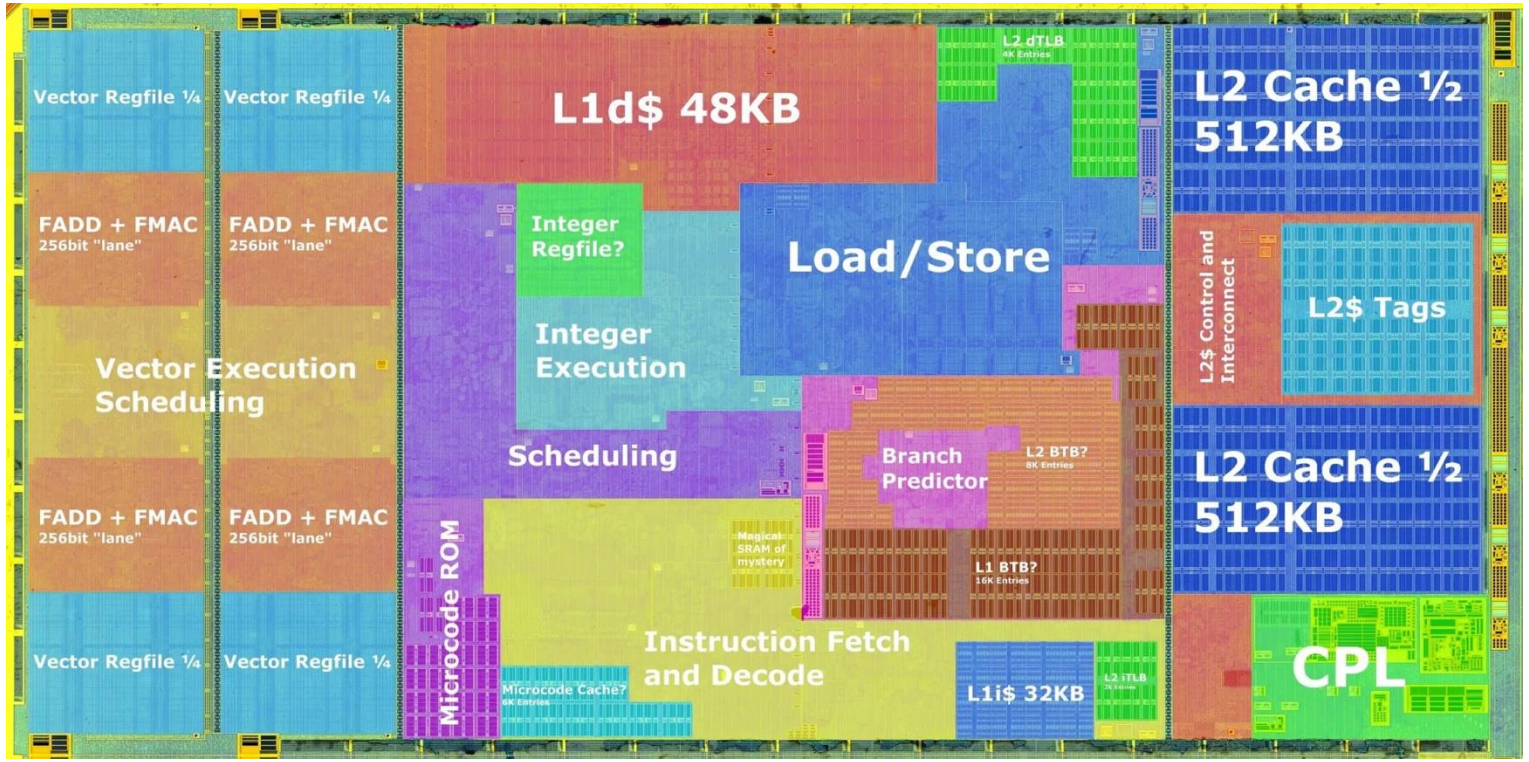
Algorithm	method	cells/gen/sec	speedup[4]
Naïve counting	count neighbor cells for every pixel	10^4	
JavaLife [35]	lookup table + linked list	$3 \cdot 10^7$	3 000 ×
QLife [67]	lookup table + quad tree	10^8	3 ×
LifeCube [62]	halfadder logic	$2 \cdot 10^9$	20 ×
Optimized LifeCube	low level assembly + stagger step + 64-ary tree	10^{10}	5 ×
GPU LifeCube	CUDA code running on a GPU	10^{13}	1 000 ×

Other benefits

- Data flow is much easier to follow
- The code and its data flow are in one place
  - You can still use sub methods
  - static method calls are free (literally take 0 time)
  - CPU does not pull in unrelated data

With traditional OOP

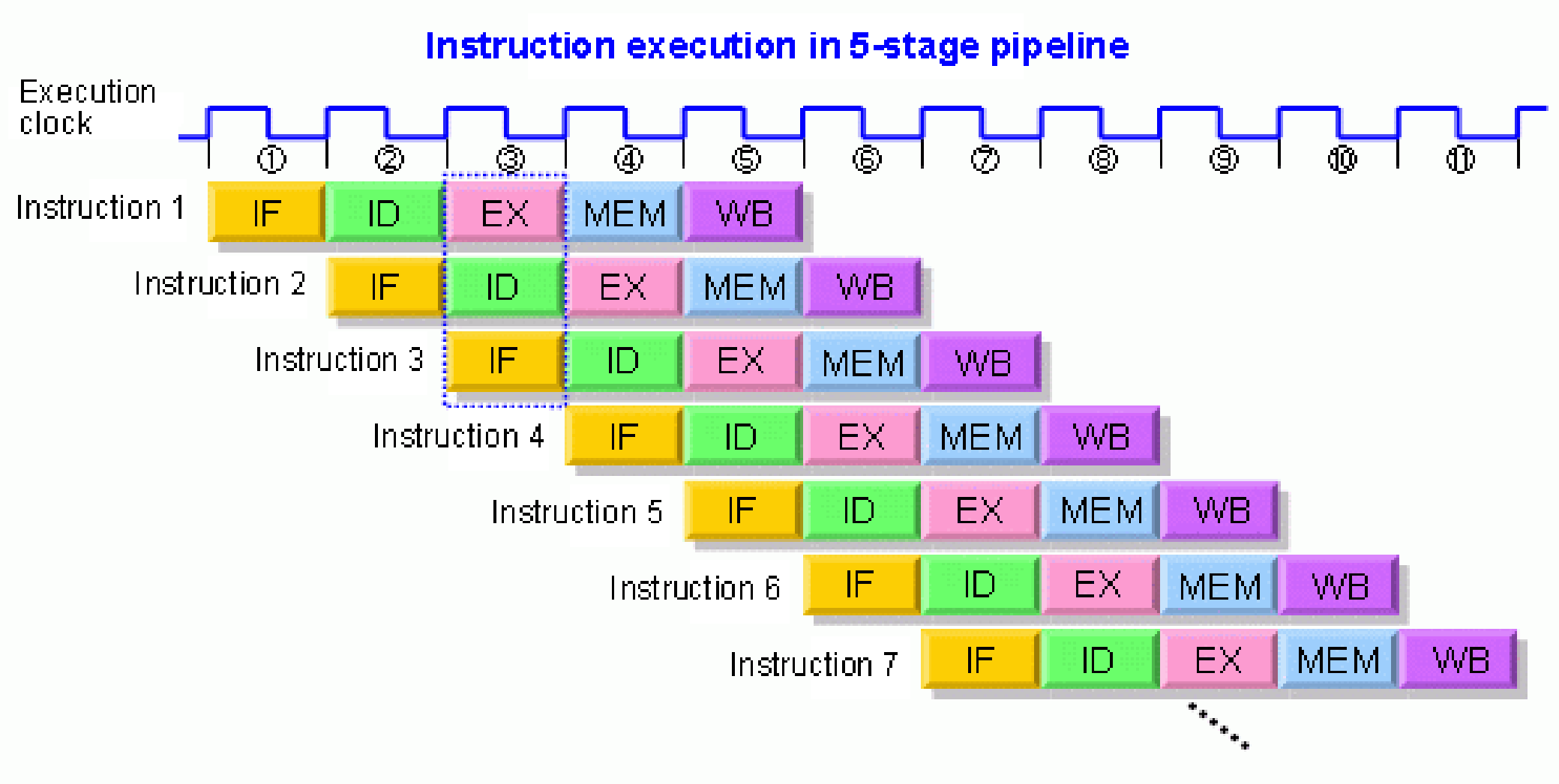
- Pacman/ghost code spread over many objects
- All member data in accessed objects gets pulled in
  - Even if never used
  - More cache eviction



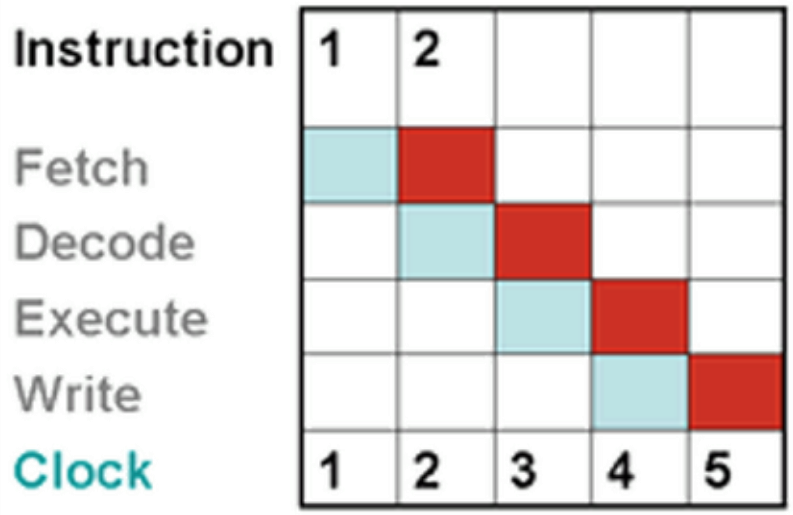
ASD 05
Data oriented design

- Demo
- Overview
- CPU internals: data flow
- Objects data flow
- CPU internals: code flow

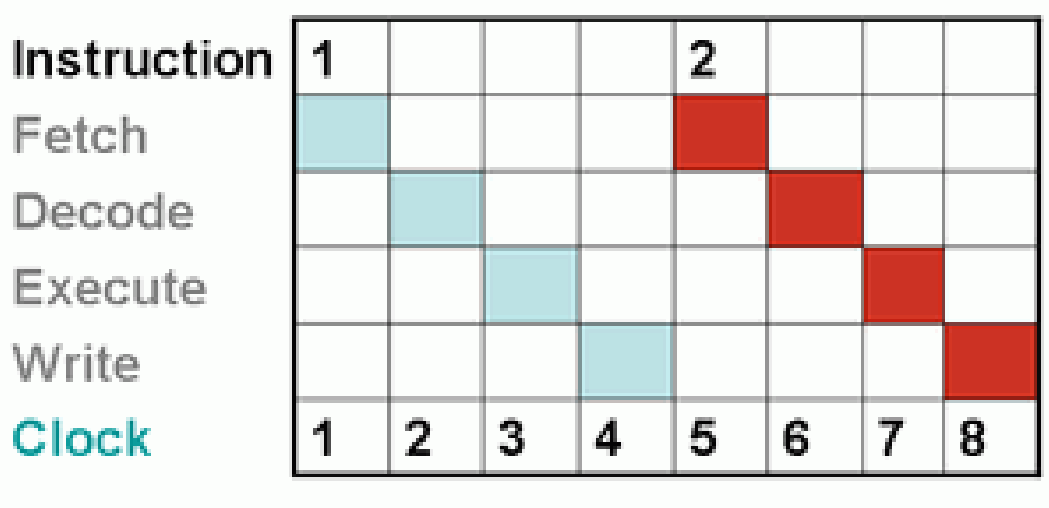
CPU code flow: pipelining



CPU code flow: pipelining



Linear code: 1 instruction every cycle



Jump: pipeline empties

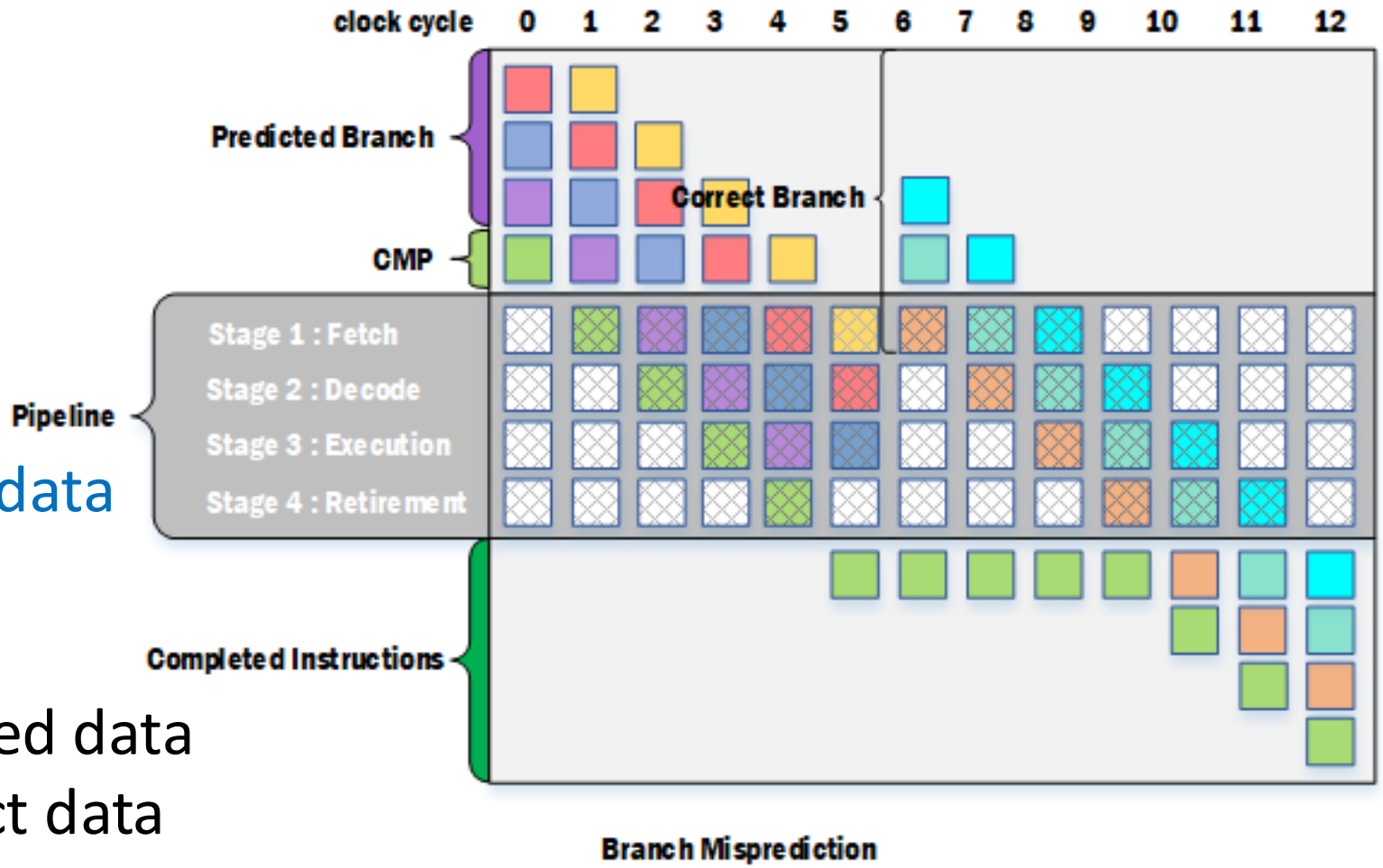
Stalls for `length(pipeline)` cycles

https://stackpointer.io/hardware/how-pipelining-improves-cpu-performance/113/

CPU prediction, code + data

CPU guesses the branch:

- Speculatively fetches and executes code, as-if guess is correct
- If correct: - commit work
- If wrong: - roll back work
  - empty pipeline
  - fetch other code



Branch Misprediction

CPU also speculatively fetches data

- If wrong
  - CPU may evict wrong data
  - may have to re-fetch evicted data
  - will have to wait for correct data

OOP problems

Virtual methods

- Essentially random to the CPU
- Cannot be predicted
- Often not in the L1_code cache
  - Costly cache wait

Heap based objects

- Stored in chaotic heap -> random locations
- CPU cannot predict data flow
- Very costly L1/L2/L3 cache misses

CPU superscalar

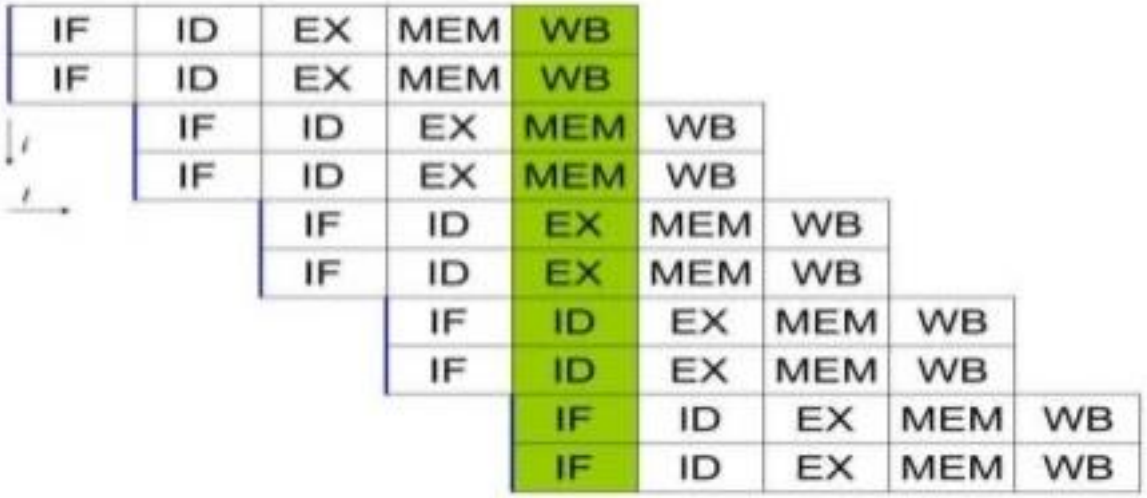
CPU analyses code

```
a = b + 10; //a depends on b
c = d * 11; //c depends on d, but not a or b
```

line 1 and 2 can execute at the same time = superscalar.

But…

- both lines run in the same pipeline



CPU out of order execution (OoO)

CPU analyses code using a dependency graph

```
1: a = b + 10; //a depends on b
2: a += d;     //must wait until b processed
3: c = d * 11; //c depends on d, but not a or b
4: c += b;     //must wait until d processed
```

CPU can reorder this to

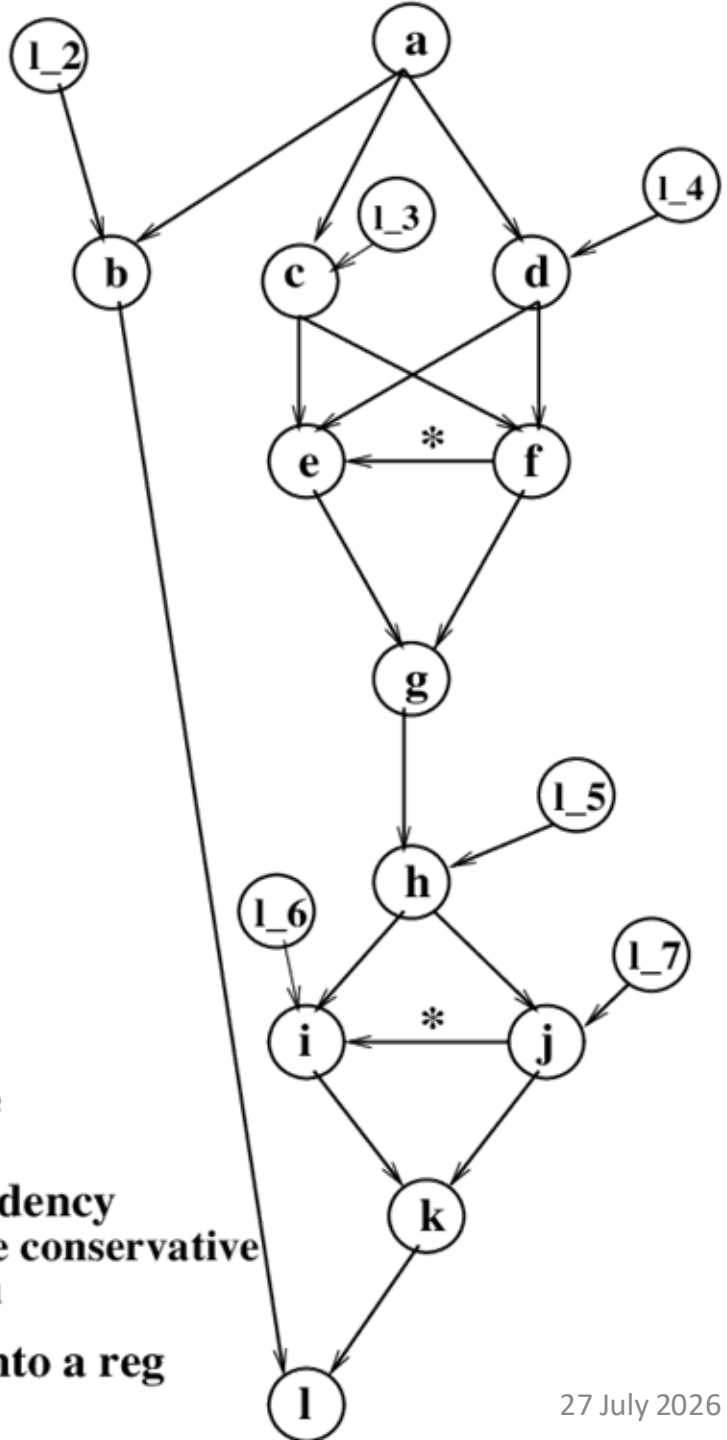
```
1: a = b + 10; //a depends on b
3: c = d * 11; //c depends on d, but not a or b
2: a += d;     //must wait until b processed
4: c += b;     //must wait until d processed
```

2-way superscalar executes 1+3, then 2+4

Example dependency graph

```
load a;
b = 2 * a;
c = a + 3;
d = a – 4;
e = c * d;
f = c – d;
g = e /  f;
h = g + 5;
i  = h * 6;
j  = h + 7;
k = i  /   j;
l  = b + k;
```

Source code



dependency edge

* additional dependency due to dependence conservative chains formation

l_x constant x loaded into a reg

Stack engine

Modern CPUs have a stack engine *(Intel P6b in 2003)*

- Simulates a stack data structure in CPU registers
- Local variables take zero time to access

Old CPUs: -> stack bookkeeping has overhead

New CPUs: -> all stack work takes zero time

C++ Objects live on the stack by default

For small_ish objects (< 8 KB)

- Zero access time, even for random access

What about the compiler?

CPU can see both instructions + the current data

- Extra info permits better **local** dependency graph
- CPU knows stuff that the compiler does not
- CPU uses neural network for predictions
  - Neural net gets trained as the program runs

Compiler sees only static picture of code

- Cannot make assumptions, or subtle bugs would happen
  - Many seemingly obvious optimisations not used
- Compiler cannot see data, only code
- Global dependency tracking is a NP-hard problem